TD3 : Système de gestion mémoire

1 Traduction d'adresses

On considère une machine avec une mémoire physique pouvant contenir (en plus du SE) 12 cases de 4ko. Deux processus P1 et P2 sont chargés en mémoire. Les tables des pages pour ces deux processus sont :

	n° de page	0	1	2	3	4	5	6	7	8	9	...
P1	n° de case	2	3	4	34	6	7	21	1	-	-	...
	Etat	R	R	R	SR	RW	RW	SRW	RW	N	N	...

	n° de page	0	1	2	3	4	5	6	7	8	9	...
P2	n° de case	2	3	4	34	8	9	0	10	11	-	...
	Etat	R	R	R	SR	RW	RW	RW	RW	RW	N	...

La ligne état contient des bits décrivant l'état de la page :

- R : page accessible en lecture seule ;
- RW : page accessible en lecture et écriture ;
- N : page invalide et non allouée ;
- S : page invalide en zone de swap (sur le disque), dans ce cas le n° de case indique le numéro du bloc (sur le disque) contenant la page (en réalité ce numéro de bloc figure dans une autre table).

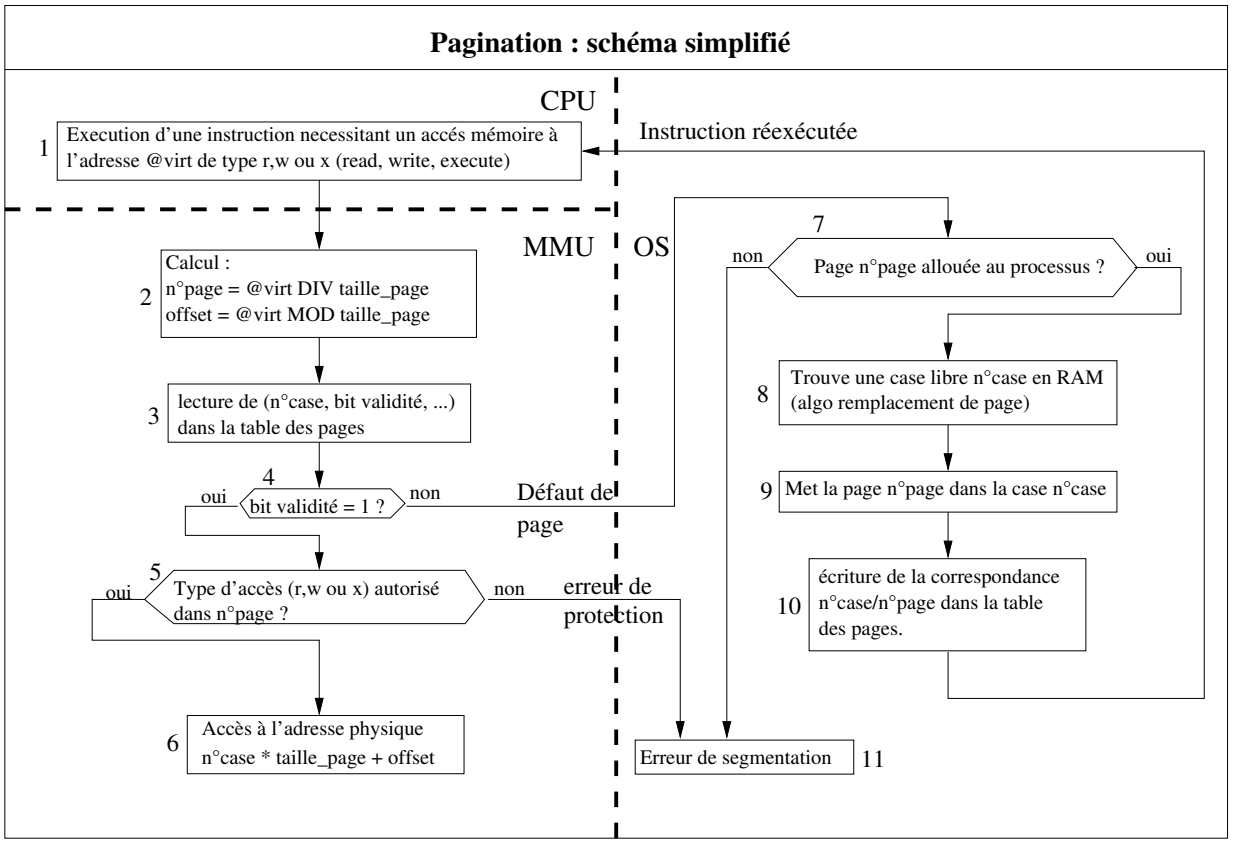
Toutes les pages au delà de la 9 ont un bit N.

Question 1 : Indiquez sur un schéma le contenu de chacune des 12 cases de la mémoire physique (i.e., les pages qu'elles contiennent). Quelles cases sont partagées ? Quelle peut être leur utilité ? Comment interprétez vous leurs bits d'état ? Quelles cases sont libres ?

Question 2 : Traduire les adresses virtuelles suivantes en adresses physique (indiquées en hexadécimal).

- Pour le processus P1 : $44EF$, $3CC$ et $700B$
- Pour le processus P2 : $44EF$, $3CC$ et $700B$

2 Pagination



Question 3 : On reprend les même processus et les mêmes tables de pages que dans l'exercice précédent. Que se passe-t-il si le processus P1 fait les accès suivant (toutes les adresses sont virtuelles) : lecture à l'adresse $2EE4$, écriture à l'adresse $122A$, lecture à l'adresse $6EA7$, lecture à l'adresse $334F$ et enfin écriture à l'adresse $83A3$. Indiquez pour chaque accès, les étapes effectuées dans le schéma et ce qui ce passe lors de chacune de ces étapes.

Question 4 : Dans le schéma précédent à quelles étapes les bits d'accès et de modification sont-ils modifiés ou utilisés ?

Question 5 : Lors de l'étape 5, comment la MMU vérifie-t-elle que l'accès est autorisé ?

Question 6 : Quelles sont les étapes où il est fait un accès à la RAM ? Et un accès au disque ?

3 Table des pages à 2 niveaux

Un système utilise des pages de 4ko. Les adresses virtuelles sont codées sur 32 bits.

Question 7 : Quel est la taille de l'espace d'adressage virtuel de chaque processus ?

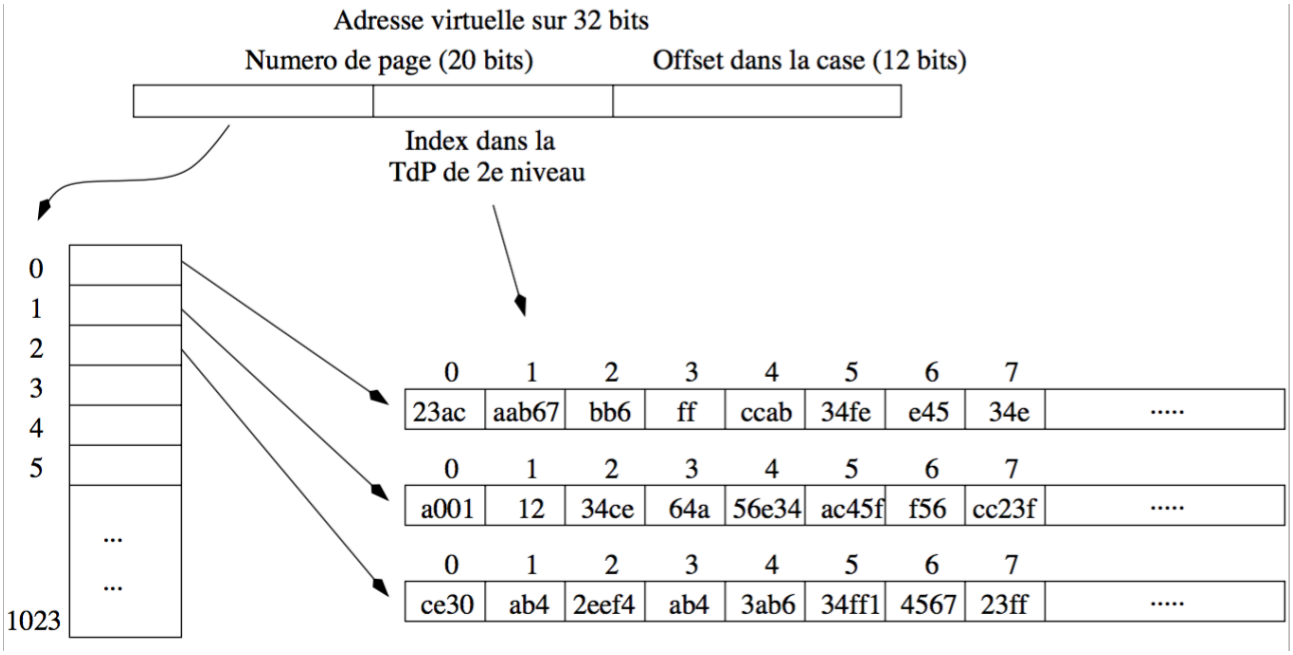
Question 8 : Combien de place prend la table des pages de chaque processus en mémoire en supposant que chaque entrée de la table utilise 4 octets (20 bits pour le numéro de la case et 12 bits pour l'état) ?

Cette taille est bien trop importante (sur un système linux classique avec 1 seul utilisateur il y a facilement 50 processus). De plus les processus n'utilisent en général qu'une toute petite partie de leur espace d'adressage virtuel, une grande partie des pages n'est donc pas allouée au processus. Pour résoudre ce problème, on utilise des tables des pages à plusieurs niveaux (2 ou 3, ici 2).

Le numéro d'une page (sur 20 bits) est découpé en deux parties de 10 bits. La première sert d'index dans une table contenant $2^{10} = 1024$ entrées. Chaque entrée de cette table de premier niveau contient l'adresse d'une table des pages de second niveau. La seconde partie du numéro de page sert d'index dans cette table des pages de second niveau qui contient le numéro de case et les bits d'état.

Question 9 : Avec la table suivante (les bits d'états des pages ne sont pas indiqués), traduire les adresses virtuelles $802EF5$, $402AAF$, 5003 et 403345.

Question 10 : Quelle va être la taille de cette table des pages pour un processus utilisant 100ko de mémoire ? 1Mo de mémoire ? 10Mo de mémoire ?



4 Utilisation du swap

On suppose que le temps de lecture d'un octet en mémoire physique est de 80ns (80.10^{-9}s). Le temps de chargement ou d'écriture d'une page depuis la zone de swap (en cas de défaut de page) est de 10ms.

Question 11 : Un processus fait 1 milliard d'accès à la mémoire lors de son exécution. Combien de temps utilise-t-il pour ses accès mémoire ?

Question 12 : Même question si 0.01% de ces accès déclenchent un défaut de page.

Question 13 : Quel doit être le taux maximum de défauts de pages si on veut que le temps d'exécution augmente de moins de 10% du fait des défauts de pages ?

5 Divers

Question 14 : Un système utilise des adresses virtuelles sur 32 bits. La traduction se fait en utilisant une table des pages à 2 niveaux. Une adresse virtuelle est donc découpée en trois champs : un champ de 9 bits pour le premier niveau, un champ de 10 bits pour le second et enfin un déplacement (offset).

Combien y a-t-il de pages ? de quelle taille ? Quelle est la taille de la table des pages de premier niveau (chaque entrée fait 4 octets) ? et celles de second niveau ?

Question 15 : Un système utilise des adresses virtuelles sur 48 bits et des adresses physiques sur 32 bits. Les pages font 8ko. Combien y a-t-il d'entrées dans la table des pages (un seul niveau) ?

Question 16 : Une instruction en langage machine charge un mot de 32 bits dans un registre du processeur (ex : `load 12345, R1`). Cette instruction est codée sur 6 octets. Combien de défauts de page l'exécution de cette instruction peut-elle déclencher au maximum ?

6 Algos de remplacement de pages

Simuler les algorithmes de remplacement de pages :

- Algorithme FIFO :
  - Avec une mémoire contenant 3 cases :

0	1	2	3	0	1	4	0	1	2	3	4

  - Avec une mémoire contenant 4 cases :

0	1	2	3	0	1	4	0	1	2	3	4

- Toujours FIFO avec mémoire de 4 cases :

0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	1	3	4	1

— Algorithme LRU (moins récemment utilisé) :

0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	1	3	4	1

— Algorithme de l'horloge :

0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	1	3	4	1

— Algorithme optimal :

0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	1	3	4	1

7 Faille

Un ancien système d'exploitation utilisait la pagination et permettait aux processus de fournir une fonction qui était exécuté lors de chaque défaut de page.

L'accès aux fichiers était protégé par un mot de passe : si un processus voulait ouvrir un fichier, il devait fournir au système d'exploitation le mot de passe correspondant. Le système d'exploitation comparait le mot de passe fourni par le processus avec le mot de passe correct caractère par caractère en s'arrêtant dès qu'une différence était trouvée (et dans ce cas l'ouverture du fichier échouait).

Ce système présente une faille de sécurité qui permet à un processus spécialement écrit de retrouver les mots de passe rapidement.

Question 17 : Si on écrit un processus pour tester tous les mots de passe possibles en supposant qu'un mot de passe utilise 8 caractères (lettres ou chiffres) et qu'il faut 1ms au système pour tester un mot de passe, combien faut-il de temps pour tous les tester ?

Question 18 : Imaginons maintenant que la chaîne de caractère que le programme va utiliser comme mot de passe ait son premier octet à la fin d'une page n et les 7 octets suivant dans la page $n+1$. On suppose de plus que la page $n+1$ est dans le swap. Que se passe-t-il si le premier caractère de ce mot de passe est correct ? et s'il est incorrect ?

Question 19 : Comment peut-on alors trouver rapidement le premier caractère du mot de passe puis ensuite tous les autres ? Combien d'essais faut-il ?

Question 20 : Comment peut-on faire pour s'assurer que la page $n+1$ est bien dans le swap ?